

Artificial Neural Network (ANN) (CS643)

Lecture 05 - Parameterized learning

Dr. Abdul Basit

Associate Professor - CS & IT
University of Balochistan

September 16, 2025

- Material taken from Chapter 8 and 9 of “ Deep Learning for Computer Vision with Python” by “Dr. Adrian Rosebrock.”

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Parameterized learning

k-NN Algorithm

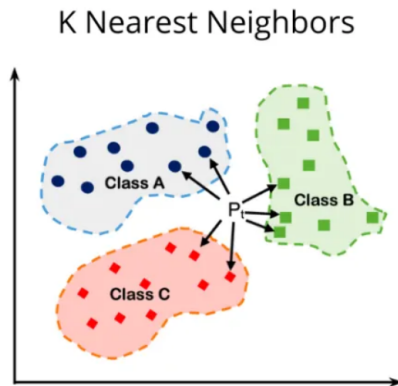


Figure: k-NN Algorithm. Source: Unknown

Parameterized learning

k-NN Algorithm

- **Measure the distance:** Calculate the distance between the new flower and all the flowers in the dataset. In our case, we can use the Euclidean distance, which is like measuring the straight-line distance between two points. The distance is computed based on the petal length and width.
- **Find the K nearest neighbors:** Identify the K flowers with the shortest distances to the new flower. These are the K nearest neighbors. For instance, if $K = 3$, we select the three flowers that are closest to our new flower.
- **Majority voting:** Among the K nearest neighbors, count how many are roses and how many are daisies. Whichever type has the majority becomes our prediction for the new flower. For example, if two neighbors are roses and one is a daisy, we predict that the new flower is a rose.

Parameterized learning

Background

- In k-NN, the training may be **simple** but testing is **quite slow**, with the bottleneck being the distance computation between vector.
- Computing the **distances** between training and testing points **scales linearly** with the number of points in our dataset, making the method **impractical** when our datasets become quite large.
- The k-NN cannot function without maintaining a **replica** of data inside the instantiation or at least have a **pointer** to training set on disk, etc.

Parameterized learning

Background

- To see why storing an exact replica of the training data inside the model is an issue, consider training a k-NN model and then deploying it to a **customer base** of 100, 1,000, or even 1,000,000 users.
- If your training set is only a few megabytes, this may not be a problem but if your training set is measured in **gigabytes to terabytes** (as is the case for many datasets that we apply deep learning to), you have a real problem on your hands.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Parameterized learning

- Instead, a more desirable approach would be to define a machine learning model that can **learn patterns** from our input data during **training time**.
- It requires us to spend more time on the training process, but have the benefit of being defined by a **small number** of parameters that can easily be used to represent the model, regardless of training size.
- This type of machine learning is called **parameterized learning**.

- **Parameterized learning:**

*“A learning model that summarizes data with a **set of parameters** of fixed size (independent of the number of training examples) is called a parametric model. No matter how much data you throw at the parametric model, it won't change its mind about how many parameters it needs.”*

- Parameterized learning is the cornerstone of modern day machine learning and deep learning algorithms.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

- **Parameterization is the process of defining the necessary parameters of a given model.**
- In the task of machine learning, parameterization involves defining a problem in terms of four key components:
 - Data
 - Scoring function
 - loss function
 - Weights and biases.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Linear classification

Data

- This component is our **input data** that we are going to learn from. This data includes both the **data points** (that is raw pixel intensities from images, extracted features, etc.) and their associated **class labels**.
- Typically we denote our data in terms of a multi-dimensional **design matrix**.

Linear classification

Data

- Each row in the **design matrix** represents a **data point** while each column (which itself could be a multi-dimensional array) of the matrix corresponds to a **different feature**.
- For example, consider a dataset of 100 images in the RGB color space, each image sized 32×32 pixels. The design matrix for this dataset would be $X \subseteq R^{100 \times (32 \times 32 \times 3)}$ where X_i defines the i -th image in R . Using this notation, X_1 is the first image, X_2 the second image, and so on.
- Along with the design matrix, we also define a **vector y** where y_i provides the **class label** for the i -th example in the dataset.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Linear classification

Scoring function

- The **scoring function** accepts our data as an input and **maps** the data to class labels.

For instance, given our set of input images, the scoring function takes these data points, applies some function f (our scoring function), and then returns the predicted class labels, similar to the pseudocode below:

```
INPUT_IMAGES => F(INPUT_IMAGES) => OUTPUT_CLASS_LABELS
```

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - **Loss function**
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Linear classification

Loss function

- A loss function quantifies how well our **predicted class labels** agree with our **ground-truth labels**. The higher level of agreement between these two sets of labels, the lower our loss (and higher our classification accuracy, at least on the training set).
- Our goal when training a machine learning model is to **minimize** the loss function, thereby increasing our classification accuracy.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - **Weights and Biases**
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Linear classification

Weights and biases

- The **weight matrix**, typically denoted as W and the **bias vector** b are called the weights or parameters of our classifier that we'll actually be **optimizing**. Based on the output of our scoring function and loss function, we will be **tweaking and fiddling** with the values of the weights and biases to increase classification accuracy.
- Once we have defined these **four key components**, we can then apply **optimization methods** that allow us to find a set of parameters W and b that minimize our loss function with respect to our scoring function (while increasing classification accuracy on our data).

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Linear classification: From images to labels

- We need our **data**. Let's assume that our training dataset is denoted as x_i where each image has an associated class label y_i . We'll assume that $i = 1, \dots, N$ and $y_i = 1, \dots, K$, implying that we have N data points of dimensionality D , separated into K unique categories.
- In the **Animals dataset**, we have $N = 3,000$ total images. Each image is 32×32 pixels, represented in the RGB color space that is three channels per image. We can represent each image as $D = 32 \times 32 \times 3 = 3,072$ distinct values. Finally, we know there are a total of $K = 3$ class labels: one for the dog, cat, and panda classes, respectively.

Linear classification: From images to labels

- Given these variables, we must now **define a scoring function f** that maps the images to the class label scores. One method to accomplish this scoring is via a simple linear mapping:

$$f(x_i, W, \mathbf{b}) = Wx_i + \mathbf{b}$$

Linear classification: From images to labels

- Let's assume that each x_i is represented as a **single column vector** with shape $[D \times 1]$ (in this example we would flatten the $32 \times 32 \times 3$ image into a list of 3,072 integers). Our weight matrix W would then have a shape of $[K \times D]$ (the number of class labels by the dimensionality of the input images).
- Finally b , the **bias vector** would be of size $[K \times 1]$. The bias vector allows us to **shift and translate** our scoring function in one direction or another without actually influencing our weight matrix W . The bias parameter is often critical for successful learning.

Linear classification: From images to labels

- Going back to the animals dataset example, each x_i is represented by a list of 3,072 pixel values, so x_i , therefore, has the shape $[3,072 \times 1]$. The weight matrix W will have a shape of $[3 \times 3,072]$ and finally the bias vector b will be of size $[3 \times 1]$.

Linear classification: From images to labels

- Figure follows an illustration of the linear classification scoring function f . On the left, we have our original input image, represented as a $32 \times 32 \times 3$ image. We then flatten this image into a list of 3,072 pixel intensities by taking the 3D array and reshaping it into a 1D list.

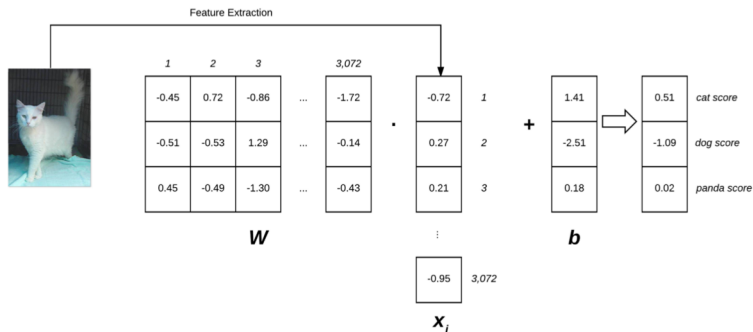


Figure: Illustrating the dot product of the weight matrix W and feature vector x , followed by the addition of the bias term. Figure inspired by Karpathy's example in Stanford University's cs231n course.

Linear classification: From images to labels

- Our weight matrix W contains three rows (one for each class label) and 3,072 columns (one for each of the pixels in the image). After taking the dot product between W and x_i , we add in the bias vector $\mathbf{b}$, the result is our **actual scoring function**.
- Our scoring function yields three values on the right: the scores associated with the dog, cat, and panda labels, respectively.

Linear classification: From images to labels

- Looking at the above figure and equation, the input x_i and y_i are fixed and not something we can modify. Sure, we can obtain different x_i s by applying various transformations to the input image but once we pass the image into the scoring function, these values do not change.
- In fact, the only parameters that we have any control over are our weight matrix W and our bias vector $\mathbf{b}$. Therefore, our goal is to utilize both our scoring function and loss function to optimize the weight and bias vectors such that our classification accuracy increases.

Linear classification: From images to labels

- Exactly how we optimize the weight matrix depends on our loss function, but typically involves some form of **gradient descent**.
- With the given a scoring function, we will also define a loss function that tells us how **good** our predictions are on the input data.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Linear classification: From images to labels

- Once we are done training our model, we can **discard** the input data and **keep** only the weight matrix W and the bias vector b . This substantially reduces the size of our model since we need to store **two sets** of vectors versus the **entire training set**.
- **Classifying new test data is fast.** In order to perform a classification, all we need to do is take the dot product of W and x_i , follow by adding in the bias b that is **apply our scoring function**. Doing it this way is significantly faster than needing to compare each testing point to every training example, as in the k-NN algorithm.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python**
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

A Simple Linear Classifier With Python

- The details of the code is in the handout.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

The role of the loss function

- In order to actually **learn** the mapping from the **input data to class labels** via our scoring function, we need to discuss two important concepts:
 - Loss functions
 - Optimization methods

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

What are loss functions?

- At the most basic level, a loss function quantifies how **good** or **bad** a given predictor is at classifying the input data points in a dataset.
- The smaller the loss, the better a job the classifier is at modeling the relationship between the input data and output class labels
- Although there is a point where we can **overfit** our model by modeling the training data too closely, our model loses the ability to generalize.
- Conversely, the larger our loss, the more work needs to be done to increase classification accuracy.

What are loss functions?

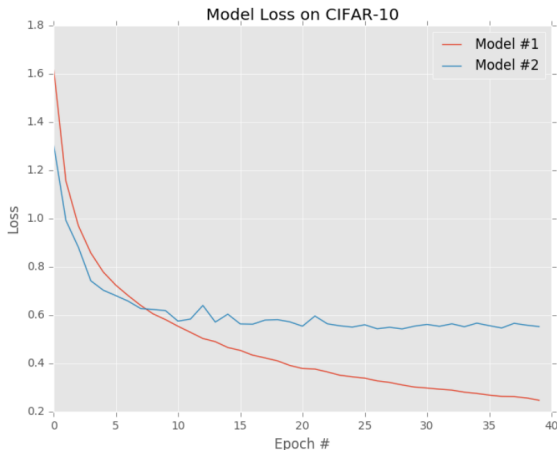


Figure: The training losses for two separate models trained on the CIFAR-10 dataset are plotted over time. Our loss function quantifies how “good” or “bad” of a job a given model is doing at classifying data points from the dataset. Model #1 achieves considerably lower loss than Model #2.

What are loss functions?

- To improve our classification accuracy, we need to **tune the parameters** of our weight matrix W or bias vector $\mathbf{b}$. Exactly how we go about updating these parameters is an **optimization problem**.
- Ideally, our loss should **decrease over time** as we tune our model parameters.
- In this specific example, Model #1 is achieving lower overall loss and is likely a more desirable model to be used on classifying other images from the CIFAR-10 dataset. I say “likely” because there is a chance that Model #1 has **overfit** to the training data.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - **Multi-class SVM loss**
 - Cross-entropy Loss and Softmax Classifiers

Multi-class SVM loss

- Multi-class **Support Vector Machines** (SVM) which uses a scoring function f to map our data points to numerical scores for each class labels. This function f is a simple learning mapping:

$$f(x_i, W, \mathbf{b}) = Wx_i + \mathbf{b}$$

- Now that we have our scoring function, we need to determine how **good** or **bad** this function is (given the weight matrix W and bias vector $\mathbf{b}$) at making predictions. To make this determination, we need a loss function.

Multi-class SVM loss

- When creating a machine learning model we have a design matrix X , where each row in X contains a **data point** we wish to classify. In the context of image classification, each row in X is an image and we seek to correctly label this image. We can access the i -th image inside X via the syntax x_i .
- Similarly, we also have a vector $\mathbf{y}$ which contains our class labels for each X . These $\mathbf{y}$ values are our **ground-truth labels** and what we hope our scoring function will correctly predict. Just like we can access a given image as x_i , we can access the associated class label via y_i .

Multi-class SVM loss

- As a matter of simplicity, we abbreviate our scoring function as s :

$$s = f(x_i, W)$$

- Which implies that we can obtain the predicted score of the j -th class via the i -th data point:

$$s_j = f(x_i, W)_j$$

- Using this syntax, we can put it all together, obtaining the **hinge loss function**:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Multi-class SVM loss

- Essentially, the hinge loss function is summing across all incorrect classes ($i \neq j$) and comparing the output of our scoring function s returned for the j -th class label (the incorrect class) and the y_i -th class (the correct class). We apply the max operation to clamp values at zero, which is important to ensure we do not sum negative values.
- A given x_i is classified correctly when the loss $L_i = 0$. To derive the loss across our entire training set, we simply take the **mean** over each **individual L_i** :

$$L = \frac{1}{N} \sum_{i=1}^N L_i$$

- Another related loss function you may encounter is the **squared hinge loss**:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)^2$$

- The squared term penalizes our loss more heavily by squaring the output, which leads to **quadratic growth** in loss in a prediction that is incorrect (versus a linear growth).
- As for which loss function you should use, that is entirely dependent on your dataset. It's typical to see the standard hinge loss function used more, but on some datasets the squared variation may obtain better accuracy. Overall, this is a **hyperparameter** that you should consider tuning.

Multi-class SVM loss

- A working example.



	Image #1	Image #2	Image #3
Dog	4.26	3.76	-2.37
Cat	1.33	-1.20	1.03
Panda	-1.01	-3.81	-2.27

Figure: The scoring function output for all three classes. Dog, cat, and panda.

Contents

- 1 Parameterized learning
 - What is parameterized learning?
- 2 Linear classification
 - Data
 - Scoring function
 - Loss function
 - Weights and Biases
- 3 Linear Classification: From Images to Labels
- 4 Advantages of parameterized learning and linear classification
- 5 A Simple Linear Classifier With Python
- 6 The role of the loss function
 - What are loss functions?
 - Multi-class SVM loss
 - Cross-entropy Loss and Softmax Classifiers

Cross-entropy Loss and Softmax Classifiers

- While **hinge loss** is quite popular, you're much more likely to run into **cross-entropy loss** and **softmax classifiers** in the context of deep learning and convolutional neural networks.
"Softmax classifiers give you probabilities for each class label while hinge loss gives you the margin."
- It's much easier for us as humans to interpret **probabilities** rather than **margin scores**. Furthermore, for datasets such as ImageNet, we often look at the **rank-5 accuracy** of Convolutional Neural Networks (where we check to see if the ground-truth label is in the top-5 predicted labels returned by a network for a given input image). Seeing if (1) the true class label exists in the top-5 predictions and (2) the probability associated with each label is a nice property.

Cross-entropy Loss and Softmax Classifiers

Understanding Cross-entropy Loss

- The **softmax classifier** is a generalization of the binary form of **Logistic Regression**. Just like in hinge loss or squared hinge loss, our mapping function f is defined such that it takes an input set of data x_i and maps them to output class labels via **dot product** of the data x_i and weight matrix W (omitting the bias term for brevity):

$$f(x_i, W) = Wx_i$$

- However, unlike hinge loss, we can interpret these scores as **unnormalized log probabilities** for each class label, which amounts to swapping out the hinge loss function with cross-entropy loss:

$$L_i = -\log(e^{s_{y_i}} / \sum_j e^{s_j})$$

Cross-entropy Loss and Softmax Classifiers

Understanding Cross-entropy Loss

- So, how did I arrive here? Let's break the function apart and take a look. To start, our loss function should minimize the negative log likelihood of the correct class:

$$L_i = -\log P(Y = y_i | X = x_i)$$

- The probability statement can be interpreted as:

$$P(Y = k | X = x_i) = e^{s_{y_i}} / \sum_j e^{s_j}$$

- Where we use our standard scoring function form:

$$s = f(x_i, W)$$

- As a whole, this yields our final loss function for a single data point, just like above:

$$L_i = -\log(e^{s_{y_i}} / \sum_j e^{s_j})$$

Cross-entropy Loss and Softmax Classifiers

Understanding Cross-entropy Loss

- Take note that your logarithm here is actually base e (natural logarithm) since we are taking the inverse of the exponentiation over e earlier. The actual exponentiation and normalization via the sum of exponents is our **Softmax function**. The negative log yields our actual **cross-entropy loss**.
- Just as in hinge loss and squared hinge loss, computing the cross-entropy loss over an entire dataset is done by taking the average:

$$L = \frac{1}{N} \sum_{i=1}^N L_i$$

Cross-entropy Loss and Softmax Classifiers

Understanding Cross-entropy Loss

- Omitting the regularization term from our loss function. We'll return to regularization, explain what it is, how to use it, and why it's critical to neural networks and deep learning in the next lecture. If the equations above seem scary, don't worry – we're about to work through numerical examples in the next section to ensure you understand how cross-entropy loss works.

Cross-entropy Loss and Softmax Classifiers

A worked softmax example

	Scoring Function
Dog	-3.44
Cat	1.16
Panda	3.91



Input Image

	Scoring Function	Unnormalized Probabilities
Dog	-3.44	0.03
Cat	1.16	3.19
Panda	3.91	49.90

Figure: **First Table:** To compute our cross-entropy loss, let's start with the output of the scoring function. **Second Table:** Exponentiating the output values from the scoring function gives us our unnormalized probabilities.

Cross-entropy Loss and Softmax Classifiers

A worked softmax example

	Scoring Function	Unnormalized Probabilities	Normalized Probabilities
Dog	-3.44	0.0321	0.0006
Cat	1.16	3.1899	0.0601
Panda	3.91	49.8990	0.9393

	Scoring Function	Unnormalized Probabilities	Normalized Probabilities	Negative Log Loss
Dog	-3.44	0.0321	0.0006	
Cat	1.16	3.1899	0.0601	
Panda	3.91	49.8990	0.9393	0.0626

Figure: Third Table: To obtain the actual probabilities, we divide each individual unnormalized probabilities by the sum of all unnormalized probabilities. **Fourth Table:** Taking the negative natural logarithm of the probability for the correct ground-truth yields the final loss for the data point.

Cross-entropy Loss and Softmax Classifiers

A worked softmax example

- Our goal is to classify whether the image above contains a dog, cat, or panda. Clearly, we can see that the image is a panda but what does our Softmax classifier think? To find out, we need to work through each of the four tables in the figure.
- The **first table** includes the output of our scoring function f for each of the three classes, respectively. These values are our unnormalized log probabilities for the three classes. **Let's exponentiate the output of the scoring function** (e^s , where s is our score function value), yielding our unnormalized probabilities (**second table**).

Cross-entropy Loss and Softmax Classifiers

A worked softmax example

- The next step is to take the unnormalized probabilities (e^{s_j}) and divide by the sum of unnormalized probabilities, thereby yielding the actual probabilities associated with each class label (**third table**). Notice how the probabilities sum to one. Finally, we can take the negative natural logarithm, $-\ln(p)$, where p is the normalized probability, yielding our final loss (the **fourth** and final table).
- In this case, our Softmax classifier would correctly report the image as panda with 93.93% confidence. We can then repeat this process for all images in our training set, take the average, and obtain the overall cross-entropy loss for the training set. This process allows us to quantify how good or bad a set of parameters are performing on our training set.

The End